

빨리 짜는 게 아니라 두 번 안 짜는 것

초효율의 원천은 타이핑 속도가 아닙니다. **중복을 안 만들고(커널), 재작업을 안 하고(스냅샷·예약), 판정이 지켜지도록 강제하는 것(게이트)** — 이 셋입니다. 오늘 GridSetting 479행이 지워질 뻔한 것도, 판정은 있었지만 **게이트가 없었기** 때문입니다.

원칙1 중복 0 — 커널에 한 번, App은 소비만

같은 기능을 F 트랙과 U 트랙에서 두 번 짜지 않는다. 커널 티켓은 이중 AC 테스트로 두 계약을 동시에 증명한다.

원칙2 재작업 0 — 쓰기 전에 계획을 확정

스냅샷 의미론(D-269), 확장 필드 선예약(context), 어댑터 인터페이스 우선. 실행 중에 대상이 움직이면 승인이 사라진다.

원칙3 판정은 게이트로 강제한다

문서에 적힌 판정은 지켜지지 않는다 — 오늘 증명됐다. 모든 판정에는 깨지면 실행이 멈추는 검사가 달려야 한다.

코딩 규약 — 계층·배치·테스트

C-1 계층 규칙 — 어디에 쓰느냐가 IP를 가른다

계층	경로	쓸 수 있는 것	금지
L3 커널	backend/kernels/{k1_event, k2_notify, k3_dashboard, k4_report, k5_trust, k6_feedback, k7_context}/	공유 로직 전부. K1~K7. 가이온 단독 IP	특정 도메인(재난) 어휘 하드코딩 금지 — 규칙은 설정으로
L4 App	backend/apps/disaster/	재난 고유 규칙·화면 조립. 커널 API 소비만	커널이 할 일을 여기서 구현 금지(중복 발생). 얇게 유지
L2 어댑터	backend/adapters/{sdn, weather, safety, satellite}/	외부 연동. 인터페이스 추상화 — 교체 가능	비즈니스 판정 금지. 어댑터는 번역만 한다
\$0.4 금지	dj-core · rj-core	—	직접 수정 절대 금지(D-207). 필요하면 후처리 계층으로 감싼다

C-2 이중 AC 테스트 — 초효율을 코드로 강제하는 장치

커널 티켓은 테스트 파일 하나에 두 계약을 함께 넣는다. 계약 AC가 먼저이고, U측 AC는 같은 테스트에 이어 붙인다. 이러면 "검수 통과 = 상용판 진척"이 자동 기록되고, 둘을 따로 찌러는 시도가 리뷰에서 바로 보인다.

```
# backend/kernels/k2_notify/tests/test_dispatch.py
def test_severe_event_dispatch():
    """[F-10 AC] 심각 등급 이벤트 발생 후 30초 내 발송 기록
    [U-2 AC] 동일 경로가 보고 자동화의 전달 채널로 재사용 가능"""
    ev = make_event(severity="severe")
    rec = notify.dispatch(ev)
    assert rec.elapsed_sec <= 30          # 계약 AC — 실패 시 검수 불가
    assert rec.channel_agnostic is True   # U측 AC — 상용판 재사용 조건
    assert rec.tenant_scoped is True      # K5 신뢰 커널 통과 증명
```

충돌 시 — 계약 AC가 우선한다(검수가 하드 데드라인). U측 AC는 계약 AC를 넘지 않는 범위에서만 확장한다 — "상용판 검증"을 핑계로 커널이 비대해지면 계약 일정이 깨진다.

C-3 불변 코딩 규칙 6개

C-3.1 커널 진입점은 전부 테넌트 스코프를 통과한다

D-105 · K5

커널의 모든 공개 함수는 @tenant_scoped() 를 거치거나, 거치지 않는 경우 **PUBLIC** 등록부에 근거와 함께 등재되어야 한다. 스코프 없이 데이터를 반환하는 커널 함수는 리뷰에서 반려한다.

C-3.2 쓰기 안전 3원칙

D-270 · 오늘 사고

① 파일 쓰기는 **임시 파일 + os.replace** 원자 교체 — 원본 truncate 금지. ② 순차 전제 명령은 **&&** 로 결합. ③ 데이터를 쓰는 스크립트는 **분류 등록부를 반드시 참조** 하고, 미참조 시 실행이 실패하도록 게이트를 건다.

C-3.3 외부 의존은 전부 타임아웃 + 폴백

W0-17 패턴

SDN·기상·위성·저장소 등 외부 호출은 **타임아웃 없는 호출** 금지. 실패 시 해당 필드만 "unavailable" 로 표시하고 나머지는 200으로 서빙한다. 외부 데이터는 **보조 신호**이지 단일 실패점이 아니다.

C-3.4 판정 로직은 코드가 아니라 설정(JSON)

계약 설계원칙③

재난 등급·임계값·융합 조건은 전부 규칙 JSON. **재기동 없이 반영**되어야 한다(F-04 AC). 새 융합 규칙 추가에 배포가 필요하면 설계가 틀린 것이다.

C-3.5 비밀은 코드·저장소 밖

D-204 · D-002

API 키·토큰은 환경변수/비밀저장소. 저장소에는 **형태만 검증**(길이·접두어)하고 실값은 넣지 않는다. .env* 에 실값 기입 금지.

C-3.6 ID·번호는 등재 전 충돌 검사

D-246 교훈

티켓·결정·커널 ID를 새로 붙일 때 **기존 정보 전수 대조**를 먼저 한다. W2-2/3/4 가 U1 티켓을 덮을 뻔한 사고의 재발 방지.

디자인 규약 — 화면·상태·역할

D-1 5상태 — 모든 화면이 예외 없이 정의한다

계약 품질기준: 정의를 100%

상태	언제	규칙
기본	정상 데이터 있음	등급 색 체계 고정: 주의=회색 · 경계=주황(#d9821e) · 심각=적색(#b5322c). 화면마다 다른 색 금지
로딩	조회 중	스켈레톤. 스피너만 두는 것 금지 — 관제 화면은 무엇이 오는지가 보여야 한다
빈	데이터 0건	"없음"이 정상인지 이상인지를 문구로 구분. "이벤트 없음(정상 감시 중)" ↔ "연결 끊김"
오류	조회 실패	부분 실패는 해당 영역만 오류 표시. 화면 전체를 오류로 덮지 않는다(저하 운전의 UI 대응)
권한없음	역할 부족	빈 화면이 아니라 "권한 필요"를 명시. 단, 존재 자체가 기밀인 데이터는 "없음"으로 — 격리 원칙 우선

D-2 역할별 뷰 — 화면을 3개 만들지 않는다

K3 프레임 원칙 — 관제요원·재난관제 관리자·기관장 화면을 따로 만들지 않는다. 같은 컴포넌트에 **역할 프로필**(표시 위젯·권한·기본 필터)을 주입해 파생시킨다. 화면이 3벌이 되면 유지보수가 3배가 되고, U3(3계층 대시보드) 재사용도 깨진다.

역할	기본 화면	차이는 프로필로만
관제요원	실시간 이벤트 + 지도 + 영상	기본 필터=미확인 이벤트 · 액션 버튼 노출(확인·종료·드론 승인)
재난관제 관리자	동일 프레임 + 설정 탭	규칙·구역·수신자 편집 권한 · SDN 연계 상태 위젯 추가
기관장/부서장	동일 프레임, 요약 모드	읽기 전용 · 집계 위젯 우선 · 개별 영상 대신 통계·보고서 진입

D-3 성능 AC — 화면 설계 단계에서 고정

항목	기준	설계 함의
이벤트 클릭 → 영상 표시	3초	썸네일 선로딩 + 스트림 사전 워밍. 클릭 후 협상 시작이면 못 맞춘다
심각 이벤트 → 알림 발송 기록	30초	발송을 동기 대기하지 않고 큐 적재 시점 을 기록. 외부 게이트웨이 지연과 분리
이벤트 API 응답	p95 500ms	목록은 커서 페이지네이션. offset 금지 — 데이터가 쌓이면 무너진다
규칙 변경 반영	재기동 0	규칙 JSON 핫리로드. 변경 감지 + 원자 교체(C-3.2)

화면에서 절대 하지 않는 것 — 원본 영상을 외부로 내보내는 경로를 만들지 않는다(계약 11조). 내보낼 수 있는 것은 **이벤트 메타데이터**와 **캡처**뿐이며, 다운로드 버튼은 반드시 권한·감사로그를 거친다.

완료 정의 · 자가검증 게이트

§1 티켓 DoD — 이 8개가 다 되어야 done

- ✓ 이중 AC 테스트 통과 — 커널 티켓은 [F-xx AC]와 [U-x AC] 두 단언이 같은 테스트에 있고 둘 다 green
- ✓ 테넌트 스코프 증명 — 새 데이터 경로가 격리 시험 대상에 등재되고 누출 0건 (미등재 상태로 done 금지)
- ✓ 5상태 정의 — 화면 티켓은 5개 상태 전부 구현·스크린샷 첨부
- ✓ 저하 운전 경로 — 외부 의존이 있으면 타임아웃·폴백 + 다운 모킹 시험 1건
- ✓ 게이트 통과 — 아래 §2의 검증 스크립트 전부 green
- ✓ 증거 파일 커밋 — docs/agent/evidence/<티켓>/ 에 실행 명령·출력·판정 근거
- ✓ 파과적 변경이면 dry-run 리포트 + 롤백 스크립트 (D-209·D-269 스냅샷 의미론)
- ✓ 보고 첫 줄에 4-value — REGISTRY v... / total=... / decisions=... / loop=... (D-240)

§2 자가검증 게이트 — 판정을 강제하는 검사들

깨지면 멈춘다

게이트	무엇을 막는가	상태
verify_kernel_map.py	커널 매핑 누락·미지 커널(K7 등)이 검증 밖으로 새는 것	가동중 D-264로 강화 완료
verify_registry.py	manifest 불일치·ID 충돌	가동중
verify_secrets.py	실값 키가 저장소로 들어가는 것	가동중 0건 유지
verify_classification.py	신설 — 데이터 쓰기 스크립트가 분류 등록부를 참조하지 않는 것 (오늘 GridSetting 사고)	즉시 신설 D-270③
verify_tenant_scope.py	스코프 없는 커널 공개 함수, PUBLIC 미등재 예외	확장 필요 C-3.1
verify_timeout.py	신설 — 타임아웃 없는 외부 호출 (AST 실측 방식, 이름 매칭 금지)	신설 C-3.3

게이트 작성 원칙 (D-264 계열) — 검증 스크립트는 "아는 것만 통과"가 아니라 "모르는 것을 만나면 멈춘다"로 작성한다. 새 커널·새 분류·새 패턴이 등장했을 때 조용히 지나가는 스크립트는 게이트가 아니라 장식이다.

§3 구현 순서 — 지금부터

- 1 백필 적용(승인 완료 · RESUME_NEXT §0) → 검증 3건 → 커밋
- 2 게이트 3종 신설·확장 — verify_classification · verify_tenant_scope 확장 · verify_timeout. 코드 구현보다 먼저 — 게이트가 없으면 이후 작업이 또 새어나간다
- 3 W0-14c 652 확대 → P0 전수 격리 시험(D-266 2종 우선) → WP-2 EXIT
- 4 EXIT 승인 후 K1 이벤트 커널부터 — 이중 AC 테스트를 먼저 쓰고 구현(테스트 우선)
- 5 K2-K6 → K3-K4 순. App(L4)은 커널이 선 뒤에 조립 — 반대로 하면 App에 로직이 고이고 중복이 생긴다

한 문장 — 이 규약의 목적은 속도가 아니라 "한 번 통과한 것이 다시 무너지지 않게" 하는 것입니다. 오늘 GridSetting 479행을 지킨 것은 판정문이 아니라 시뮬레이션이었습니다 — 앞으로는 그 자리에 게이트가 서게 하십시오.